I built LinkScope to make outage risks visible.

- I answer three operational questions, then show a short live outage.
- I explain the decisions I owned, not just the code the tool generated.
- I distinguish a verified local demo from a service ready to share.

I would investigate Berlin–Hamburg first.

- **Q1. “Where does the network have single points of failure?”**
  I evaluated all **92** single failures: **37 nodes and 6 links** meet the literal criticality rule. Every node loses its own traffic; that alone does not mean it harms others (Paris).
- I found wider impacts: losing **L12 Berlin–Hamburg** puts L26 at **128.30%**; losing **HUB-EAST** cuts off OPS-EAST-1 and loses **20 units**.
- **Q2. “Which links does the traffic depend on most?”**
  I found L12 busiest: **390.55 of 550 units, 71.01%**. Next is L3 Amsterdam–Hamburg: **376.23 units, 68.40%**.

I can show why a connected site may still lose traffic.

- **Q3. “What happens when a given link or site goes down?”**
  I show one failure at a time, separating the failed node’s own traffic from other undeliverable traffic.
- I fail **Link L12**: L26 rises to **128.30%**.
- I fail **Node Hamburg**: **27 endpoint demands, 162 units** lost; L26 reaches **117.06%**.
- I fail **Link HUB-EAST–Warsaw**: **20 units** undeliverable, without disconnection.
- I choose **Restore baseline**: L12 returns to **71.01%**, without reloading.

I kept the model independent of the screen.

Part	Organization and reason
Inputs and model	`data/`: unchanged backbone and sites; nodes, links, demands and relay roles.
Backend calculations	`topology.py` loads and validates; `routing.py` routes; `failures.py` explains failures. Pure functions isolate each request.
Routing rule	Fewest hops; equal shares across tied paths. Sites and data centres never relay. No database or baseline mutation.
Web interface	`network_api.py`: `GET /api/network` returns topology, baseline, top five and criticality. `GET /api/analysis?kind=link&id=L12` returns failure impacts.
Browser and delivery	`api.ts`, `NetworkWorkspace`, `NetworkMap`, `FailureControls`. I chose Django and React with TypeScript in one image, and a real geographic map.

I gave the tool boundaries and a gate before coding.

- I supplied `AGENTS.md` and `.github/copilot-instructions.md`: small reviewable changes, existing conventions, no secrets.
- I supplied project context and a **plan, execute, verify** workflow before implementation.
- I used milestones with acceptance gates; I reviewed each milestone and addressed findings from **6 review-agent reports**.
- I required **45 Django tests, 16 frontend tests**, continuous integration helper tests, TypeScript type checks, Ruff and Biome lint checks.
- I required development and production image builds and container web checks. **No local Git hooks**: branch protection requires the continuous integration `test` check before merging to main.

I checked the numbers against evidence outside the tool.

- I checked the brief’s totals: **37 nodes, 55 links, 391 demands, 2,068 units**.
- I used tests with hand-calculated expected values: a **6-node** merge with **5 links, 4 demands, 77 units**, plus fractional equal splits.
- I checked overload boundaries: **12 units** rerouted onto **10-unit** links; exactly **100%** is not overloaded, above it is.
- I checked traffic conservation, endpoints never relaying and input-order independence; all **92 failures** ran twice with matching results.
- I used independent evidence: an external validator reproduced failures and restore from the public image; I rebuilt the development container and started the downloaded Compose file.

I would not expose this demo as a service.

Concern	Now and scope decision	First internal release
Input validation and error handling	**Done well:** file and row checks, friendly errors, private log details, bad-input tests.	Keep these; add alerting.
Dependency and security checks	**Done well:** locked dependencies, lint, tests, image checks, non-root container, merge gate.	Add static security analysis, dependency and image vulnerability scans.
Authentication and access control	None; local demo only.	Single sign-on and role-based authorization.
Logging	Request and error logs to standard output; local visibility only.	Central collection, retention and alerts.
API versioning	None; browser and backend ship together.	Version and deprecation policy before other clients.
Configuration and secrets	Runtime secret key required; public key for local demo only.	Secret manager, rotation and encrypted web connections.

I redirected effort toward the operational question.

What the tool produced	What I changed	Why
Eleven requirements squeezed into three checklist items; most effort on infrastructure.	Milestones M6a–M6f, each with an acceptance gate.	The application is the assignment; keep it simple.
A proposed vector diagram.	A real Leaflet map with OpenStreetMap.	Operations needs geographic context.
CI on every development push.	Only main pushes, pull requests and version tags; forks blocked.	Limited automation budget.
Extra Docker stages; the app folder as the user's home.	Base, development and production stages; a separate home.	Less to maintain; no tool clutter in the source.
Multi-platform image builds.	Linux AMD64 only, emulated on my Mac.	No ARM requirement; simpler builds.

I made ambiguous requirements explicit.

What was unclear	How I resolved it
The official SNDlib download was unavailable.	I asked Quantum Sky and used an approved mirror; kept the file unchanged with checksum and attribution recorded.
The literal criticality rule marks every node critical.	I kept that rule and separated own endpoint losses from other undeliverable traffic. I would ask the panel whether that is the intended operational meaning.
Who should see bad-input file and row details?	I kept those details in server logs; the public interface and screen return a friendly generic error.

I still need to record the hours honestly.

- **Time spent: TBD.** I will fill this in before presenting.
- My history spans **October 23–27, 2026**, with **78 commits**; that is not an hours estimate.
- I would next prioritize access control, security scanning and central logging before shared use.